

Đặc điểm và ứng dụng của cây lệch trái

Huang Yuanhe

Trường Trung học số 1 thành phố Trung Sơn, Quảng Đông

Nguồn gốc: Đặc điểm và ứng dụng của cây lệch trái

IOI China candidate team papers / 2005 / Huang Yuanhe.doc

Tổng quan. Bài viết này trình bày khá chi tiết các đặc điểm của cây lệch trái (*leftist tree*) và các thao tác của nó.

Phần đầu nêu khái niệm đồng có thể gộp, chỉ ra điểm yếu của đồng nhị phân và dẫn tới cây lệch trái. Phần thứ hai giới thiệu định nghĩa và các tính chất của cây lệch trái. Phần thứ ba mô tả chi tiết các thao tác trên cây lệch trái và phân tích độ phức tạp thời gian. Phần thứ tư dùng một bài ví dụ để minh họa ứng dụng của cây lệch trái trong các kì thi tin học. Phần thứ năm so sánh nhiều loại đồng có thể gộp. Cuối cùng, bài viết tổng kết đặc điểm và triển vọng ứng dụng của cây lệch trái.

Từ khóa. Cây lệch trái; đồng có thể gộp; hàng đợi ưu tiên.

1 Mở đầu

Hàng đợi ưu tiên rất thường gặp trong thi lập trình. Trong các bài toán thống kê, bài toán cực trị, mô phỏng, tham lam, v.v., hàng đợi ưu tiên đều có phạm vi ứng dụng rộng. Đồng nhị phân là một hàng đợi ưu tiên phổ biến: dễ lập trình và hiệu quả. Tuy nhiên, nếu bài toán cần gộp hai hàng đợi ưu tiên, hiệu quả của đồng nhị phân không còn thỏa đáng. Cây lệch trái được giới thiệu trong bài viết này giải quyết tốt loại nhu cầu đó.

2 Định nghĩa và tính chất của cây lệch trái

Trước khi giới thiệu cây lệch trái, ta làm rõ khái niệm hàng đợi ưu tiên và đồng có thể gộp.

2.1 Hàng đợi ưu tiên và đồng có thể gộp

2.1.1 Định nghĩa hàng đợi ưu tiên

Hàng đợi ưu tiên (*priority queue*) là một kiểu dữ liệu trừu tượng (*abstract data type*, ADT). Nó là một vật chứa gồm một số phần tử, còn gọi là các nút (*node*) trong hàng đợi. Các nút của hàng đợi ưu tiên ít nhất phải có một thuộc tính: có thể sắp thứ tự, tức là hai nút bất kỳ có thể so sánh được với nhau. Để cụ thể, giả sử mỗi nút có một khóa (*key*); kích thước của nút được xác định bằng cách so sánh khóa. Hàng đợi ưu tiên có ba thao tác cơ bản: chèn nút (INSERT), lấy nút nhỏ nhất (MINIMUM) và xóa nút nhỏ nhất (DELETE-MIN).

2.1.2 Định nghĩa đồng có thể gộp

Đồng có thể gộp (*mergeable heap*) cũng là một kiểu dữ liệu trừu tượng. Ngoài ba thao tác cơ bản của hàng đợi ưu tiên (INSERT, MINIMUM, DELETE-MIN), nó còn hỗ trợ một thao tác bổ sung: gộp.

$$H \leftarrow \text{Merge}(H_1, H_2).$$

Hàm Merge xây dựng và trả về một đồng mới H chứa toàn bộ phần tử của H_1 và H_2 .

Như đã nói ở trên, nếu không cần thao tác gộp thì đồng nhị phân là lựa chọn lý tưởng. Đáng tiếc là việc gộp hai đồng nhị phân có độ phức tạp $O(n)$; nếu dùng nó để hiện thực đồng có thể gộp, thao tác gộp tất yếu trở thành nút thắt của thuật toán. Cây lệch trái (*leftist tree*), đồng nhị thức (*binomial heap*) và đồng Fibonacci đều là các đồng có thể gộp rất tốt. Bài viết này bàn về cây lệch trái; ở phần sau ta sẽ so sánh nhiều loại đồng có thể gộp.

2.2 Định nghĩa cây lệch trái

Cây lệch trái là một cách hiện thực đồng có thể gộp. Nó là một cây nhị phân. Ngoài hai con trỏ tới cây con trái và cây con phải (left, right) như nút cây nhị phân thông thường, mỗi nút còn có hai thuộc tính: khóa và khoảng cách (dist). Khóa dùng để so sánh các nút như ở trên. Khoảng cách được định nghĩa như sau.

Nút i được gọi là nút ngoài (*external node*) khi và chỉ khi cây con trái hoặc cây con phải của nó rỗng:

$$\text{left}(i) = \text{NULL} \quad \text{hoặc} \quad \text{right}(i) = \text{NULL}.$$

Khoảng cách của nút i , ký hiệu $\text{dist}(i)$, là số cạnh trên đường đi từ i tới nút ngoài gần nhất trong các hậu duệ của nó. Đặc biệt, nếu bản thân i là nút ngoài thì khoảng cách của nó bằng 0. Khoảng cách của nút rỗng được quy ước là -1 :

$$\text{dist}(\text{NULL}) = -1.$$

Khi nói khoảng cách của một cây lệch trái, ta hiểu đó là khoảng cách của gốc cây.

Cây lệch trái thỏa hai tính chất cơ bản sau.

Tính chất 1. Khóa của một nút không lớn hơn khóa của các nút con trái và phải của nó. Nói cách khác, với mỗi nút con i ,

$$\text{key}(\text{parent}(i)) \leq \text{key}(i).$$

Tính chất này còn gọi là tính chất đồng. Cây thỏa tính chất này là cây có thứ tự đồng (*heap-ordered*). Nhờ tính chất 1, gốc của cây lệch trái là nút nhỏ nhất của toàn cây, nên thao tác lấy nút nhỏ nhất chạy trong $O(1)$.

Tính chất 2. Khoảng cách của nút con trái không nhỏ hơn khoảng cách của nút con phải:

$$\text{dist}(\text{left}(i)) \geq \text{dist}(\text{right}(i)).$$

Đây là tính chất lệch trái. Tính chất 2 giúp duy trì tính chất đồng với chi phí nhỏ hơn sau hai thao tác cơ bản còn lại của hàng đợi ưu tiên: chèn nút và xóa nút nhỏ nhất.

Hai tính chất trên áp dụng cho mọi nút, vì thế hai cây con trái và phải của một cây lệch trái cũng là cây lệch trái. Từ đó ta có thể định nghĩa gọn: *cây lệch trái là cây nhị phân có thứ tự đồng và có tính chất lệch trái*.

Một cây lệch trái có thể được ghi bằng cách gắn mỗi nút với cặp (key, dist). Sơ đồ dưới đây minh họa ý tưởng đó: lá có khoảng cách 0, còn các nút nằm sâu hơn vẫn thỏa khóa cha không lớn hơn khóa con và khoảng cách cây con trái không nhỏ hơn cây con phải.

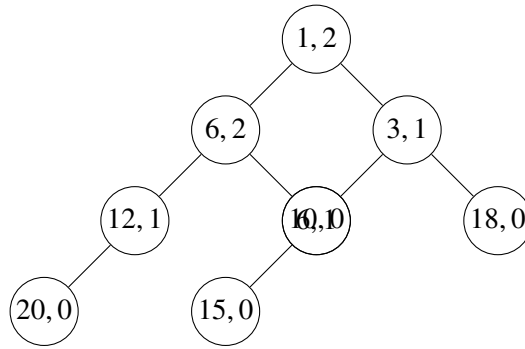


Figure 1: Ví dụ cây lệch trái: mỗi nút ghi (key, dist).

2.3 Các tính chất khác

Ta đã giới thiệu hai tính chất cơ bản. Dưới đây là hai tính chất nữa của cây lệch trái.

Một nút phải đi qua nút con của nó mới tới được nút ngoài. Vì tính chất 2, khoảng cách của một nút thực chất là số cạnh khi đi liên tục theo nhánh phải cho tới một nút ngoài.

Tính chất 3. Khoảng cách của một nút bằng khoảng cách của nút con phải cộng 1:

$$\text{dist}(i) = \text{dist}(\text{right}(i)) + 1.$$

Nút ngoài có khoảng cách 0; theo tính chất 2, nút con phải của nó phải rỗng. Để công thức trên vẫn đúng, ta đã quy ước khoảng cách của nút rỗng là -1 .

Ta thường nghĩ cây cân bằng có độ sâu rất nhỏ, tức là đi tới bất kỳ nút nào cũng cần ít cạnh. Cây lệch trái không được thiết kế để truy cập nhanh mọi nút. Mục đích của nó là truy cập nhanh nút nhỏ nhất và nhanh chóng khôi phục tính chất đồng sau khi sửa cây. Nó không cân bằng; do tính chất 2, cấu trúc nghiêng về bên trái. Tuy nhiên khái niệm khoảng cách khác với độ sâu của cây: cây lệch trái không có nghĩa là số nút hoặc độ sâu của cây con trái nhất định lớn hơn cây con phải.

Ta xét quan hệ giữa khoảng cách và số nút.

Bổ đề 1. Nếu khoảng cách của cây lệch trái là một giá trị cố định, thì cây lệch trái có ít nút nhất là cây nhị phân đầy đủ.

Chứng minh. Theo tính chất 2, số nút là nhỏ nhất khi và chỉ khi với mọi nút i trong cây đều có

$$\text{dist}(\text{left}(i)) = \text{dist}(\text{right}(i)).$$

Rõ ràng cây nhị phân có tính chất này là cây nhị phân đầy đủ.

Định lý 1. Nếu một cây lệch trái có khoảng cách k , thì cây đó có ít nhất

$$2^{k+1} - 1$$

nút.

Chứng minh. Theo bổ đề 1, trường hợp ít nút nhất là cây nhị phân đầy đủ. Cây nhị phân đầy đủ có khoảng cách k cũng có chiều cao k , nên có $2^{k+1} - 1$ nút. Vì vậy mọi cây lệch trái có khoảng cách k đều có ít nhất $2^{k+1} - 1$ nút.

Hệ quả là:

Tính chất 4. Một cây lệch trái có N nút có khoảng cách nhiều nhất

$$\lfloor \log(N + 1) \rfloor - 1.$$

Chứng minh. Giả sử khoảng cách của cây là k . Theo định lý 1,

$$N \geq 2^{k+1} - 1,$$

nên

$$k \leq \lfloor \log(N + 1) \rfloor - 1.$$

Với bốn tính chất trên, ta bắt đầu bàn về các thao tác của cây lệch trái.

3 Các thao tác trên cây lệch trái

Phần này thảo luận các thao tác trên cây lệch trái: chèn nút mới, xóa nút nhỏ nhất, gộp cây lệch trái, xây dựng cây lệch trái và xóa một nút đã biết. Vì mọi thao tác đều dựa vào thao tác gộp, ta xét thao tác gộp trước.

3.1 Gộp hai cây lệch trái

$$C \leftarrow \text{Merge}(A, B).$$

Thao tác Merge gộp hai cây lệch trái A và B , trả về một cây lệch trái mới C chứa mọi phần tử của A và B . Trong bài này, một cây lệch trái được biểu diễn bằng con trỏ tới nút gốc của nó.

Trường hợp đơn giản nhất là một trong hai cây rỗng, tức là con trỏ gốc bằng NULL. Khi đó chỉ cần trả về cây còn lại.

Nếu A và B đều không rỗng, giả sử gốc của A không lớn hơn gốc của B ; nếu không thì hoán đổi A và B . Lấy gốc của A làm gốc của cây mới C . Phần việc còn lại là gộp cây con phải của A với B :

$$\text{right}(A) \leftarrow \text{Merge}(\text{right}(A), B).$$

Sau khi gộp $\text{right}(A)$ và B , khoảng cách của $\text{right}(A)$ có thể tăng. Nếu khoảng cách này lớn hơn khoảng cách của $\text{left}(A)$, tính chất 2 bị phá vỡ; khi đó chỉ cần hoán đổi hai cây con trái và phải của A . Cuối cùng, vì khoảng cách của cây con phải có thể đã thay đổi, ta cập nhật:

$$\text{dist}(A) \leftarrow \text{dist}(\text{right}(A)) + 1.$$

Không khó kiểm tra rằng cây C sau khi gộp thỏa cả tính chất 1 và tính chất 2, do đó là cây lệch trái.

Quá trình trên có thể mô tả bằng mã giả:

```
Function Merge(A, B)
  If A = NULL Then return B
  If B = NULL Then return A
  If key(B) < key(A) Then swap(A, B)
  right(A) <- Merge(right(A), B)
  If dist(right(A)) > dist(left(A)) Then
    swap(left(A), right(A))
  If right(A) = NULL Then dist(A) <- 0
  Else dist(A) <- dist(right(A)) + 1
  return A
End Function
```

Ta phân tích độ phức tạp thời gian của thao tác gộp. Mỗi bước đệ quy đều tách một trong hai cây và luôn đưa cây con phải vừa tách vào lần gộp tiếp theo. Theo tính chất 3, khoảng cách của một cây được quyết định bởi khoảng cách cây con phải, và khoảng cách cây con phải giảm sau mỗi lần tách. Vì thế số lần mỗi cây A hoặc B bị tách không vượt quá khoảng cách tương ứng của nó. Theo tính chất 4, số lần tách không vượt quá

$$\lfloor \log(N_1 + 1) \rfloor + \lfloor \log(N_2 + 1) \rfloor - 2,$$

trong đó N_1, N_2 lần lượt là số nút của A, B . Do đó độ phức tạp xấu nhất của thao tác gộp là

$$O(\log N_1 + \log N_2).$$

3.2 Chèn nút mới

Cây chỉ gồm một nút chắc chắn là cây lệch trái. Vì vậy chèn một nút vào cây lệch trái có thể xem như gộp hai cây lệch trái:

```
Procedure Insert(x, A)
  B <- MakeIntoTree(x)
  A <- Merge(A, B)
End Procedure
```

Vì một trong hai cây được gộp chỉ có một nút, độ phức tạp thời gian của thao tác chèn là $O(\log n)$.

3.3 Xóa nút nhỏ nhất

Theo tính chất 1, gốc của cây lệch trái là nút nhỏ nhất. Sau khi xóa gốc, hai cây con còn lại đều là cây lệch trái và cần được gộp lại. Mã giả rất đơn giản:

```
Function DeleteMin(A)
  t <- key(root(A))
  A <- Merge(left(A), right(A))
  return t
End Function
```

Sau khi xóa nút nhỏ nhất chỉ cần một lần gộp, nên độ phức tạp thời gian cũng là $O(\log n)$.

3.4 Xây dựng cây lệch trái

Xây dựng một cây lệch trái từ n nút cũng là một thao tác thường dùng.

Thuật toán 1. Cách trực tiếp là chèn từng nút một. Độ phức tạp thời gian là $O(n \log n)$.

Thuật toán 2. Mô phỏng cách xây dựng đồng nhị phân:

1. Đưa n nút vào một hàng đợi FIFO, mỗi nút được xem là một cây lệch trái riêng.
2. Lặp lại: lấy hai cây lệch trái ở đầu hàng đợi, gộp chúng rồi đưa kết quả vào cuối hàng đợi.
3. Khi hàng đợi chỉ còn một cây lệch trái, thuật toán kết thúc.

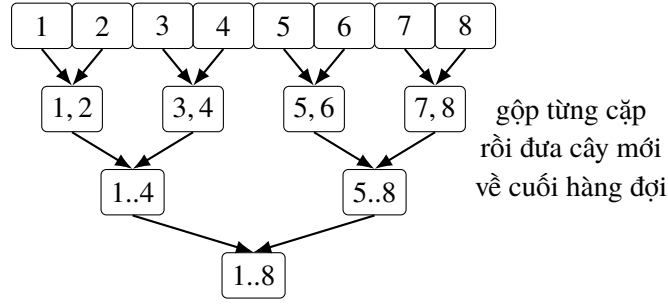


Figure 2: Xây cây lệch trái bằng cách gộp cặp đôi các cây có kích thước gần nhau.

Quá trình gộp cặp đôi biến các cây một nút thành các cây lớn hơn. Về bản chất, các cây có kích thước gần nhau được gộp theo từng tầng, tương tự một lịch gộp cân bằng.

Ta phân tích độ phức tạp của thuật toán 2. Giả sử $n = 2^k$. Khi đó:

$n/2$	lần đầu	gộp hai cây có 1 nút,
$n/4$	lần tiếp theo	gộp hai cây có 2 nút,
$n/8$	lần tiếp theo	gộp hai cây có 4 nút,
	...	
$n/2^i$	lần tiếp theo	gộp hai cây có 2^{i-1} nút.

Gộp hai cây lệch trái có 2^i nút tốn $O(i)$. Vì thế tổng thời gian là

$$\frac{n}{2}O(1) + \frac{n}{4}O(2) + \frac{n}{8}O(3) + \dots = O\left(n \sum_{i=1}^k \frac{i}{2^i}\right) = O(n).$$

3.5 Xóa một nút đã biết bất kỳ

Tiếp theo là thao tác xóa một nút đã biết bất kỳ. Cần nhấn mạnh chữ “đã biết” vì nút ở đây không được tìm theo khóa. Ngoài việc nhanh chóng tìm nút nhỏ nhất, cây lệch trái không thể tìm hiệu quả một nút có khóa chỉ định. Vì thế ta không thể yêu cầu: hãy xóa mọi nút có khóa bằng 100.

Như đã nói, hàng đợi ưu tiên là một vật chứa. Với vật chứa thông thường, một khi nút được đưa vào, vật chứa sở hữu hoàn toàn nút đó; mỗi nút trong vật chứa có một đối tượng duy nhất nắm quyền sở hữu (*ownership*). Trong cách dùng này, hàng đợi ưu tiên chỉ có thể xóa nút nhỏ nhất, vì ta không biết các nút khác là nút nào.

Tuy nhiên, ngoài vai trò vật chứa, hàng đợi ưu tiên còn có tác dụng tìm nút nhỏ nhất. Nhiều ứng dụng nhắm vào chức năng này: chúng không chuyển toàn bộ quyền sở hữu cho hàng đợi ưu tiên, mà dùng hàng đợi ưu tiên như một bộ chọn nút nhỏ nhất, lần lượt chọn các nút từ một tập. Khi đó quyền sở hữu của một nút cũng có thể đồng thời nằm ở các đối tượng khác. Nói cách khác, một nút tuy không phải nút nhỏ nhất và không thể được “biết” từ hàng đợi ưu tiên, nhưng vẫn có thể được “biết” từ một chủ sở hữu khác.

Cách dùng hàng đợi ưu tiên như vậy rất thường gặp. Hãy tưởng tượng một đồng hồ báo thức ghi nhiều thời điểm báo; vì thời gian là tuyến tính, chuông chỉ có thể reo lần lượt, nên hàng đợi ưu tiên rất thích hợp để chọn thời điểm kế tiếp. Mặt khác, người dùng phải có thể hủy bất cứ lúc nào một thời điểm báo đã biết. Việc đó cần thao tác xóa nút đã biết bất kỳ.

Thao tác xóa này cần chỉ định nút bị xóa, và tương thích với thao tác xóa gốc trước đó, vì gốc chắc chắn là một nút đã biết. Sau khi xóa một nút x , còn lại hai cây con của nó; cả hai đều là cây lệch trái. Trước hết gộp chúng thành một cây lệch trái mới:

$$p \leftarrow \text{Merge}(\text{left}(x), \text{right}(x)).$$

Bây giờ p trở tới cây mới. Nếu nút bị xóa là gốc, công việc kết thúc. Nếu x không phải gốc thì phức tạp hơn: khoảng cách của cây mới p có thể lớn hơn hoặc nhỏ hơn khoảng cách cũ của x . Điều này có thể ảnh hưởng tới khoảng cách của cha cũ q , vì q giờ trở thành cha của p . Ta phải làm như trong thao tác gộp: điều chỉnh hai cây con của q và cập nhật khoảng cách của q . Việc này tạo phản ứng dây chuyền; ta đi dọc chuỗi cha của q lên trên để điều chỉnh.

Nếu $\text{dist}(p) + 1$ bằng khoảng cách cũ $\text{dist}(q)$, thì dù p là cây con trái hay phải của q , không cần điều chỉnh q ; thao tác xóa kết thúc.

Nếu $\text{dist}(p) + 1 < \text{dist}(q)$, thì khoảng cách của q phải đổi thành $\text{dist}(p) + 1$. Nếu p là cây con trái, khoảng cách cây con trái của q nhỏ hơn cây con phải, nên phải hoán đổi hai cây con. Vì khoảng cách của q giảm, cha của q cũng phải xử lý tương tự.

Trường hợp còn lại là khoảng cách của p tăng, làm $\text{dist}(p) + 1 > \text{dist}(q)$. Nếu p là cây con trái, khoảng cách của q không đổi và thao tác xóa có thể kết thúc. Nếu p là cây con phải, có hai khả năng. Nếu khoảng cách của p vẫn không lớn hơn khoảng cách cây con trái của q , ta chỉ cần chỉnh khoảng cách của q . Nếu khoảng cách của p lớn hơn khoảng cách cây con trái của q , cần hoán đổi hai cây con của q và cập nhật khoảng cách. Sau hoán đổi, cây con phải của q là cây con trái cũ; khoảng cách của nó cộng 1 chỉ có thể bằng hoặc lớn hơn khoảng cách cũ của q . Nếu bằng, thao tác xóa kết thúc; nếu không, khoảng cách của q tăng và ta tiếp tục xử lý cha của q .

Mã giả cho thao tác xóa nút đã biết:

```

Procedure Delete(x)
  q <- parent(x)
  p <- Merge(left(x), right(x))
  parent(p) <- q
  If q != NULL and left(q) = x Then
    left(q) <- p
  If q != NULL and right(q) = x Then
    right(q) <- p
  While q != NULL Do
    If dist(left(q)) < dist(right(q)) Then
      swap(left(q), right(q))
    If dist(right(q)) + 1 = dist(q) Then
      Exit Procedure
    dist(q) <- dist(right(q)) + 1
    p <- q
    q <- parent(q)
  End While
End Procedure

```

Ta xét độ phức tạp thời gian theo hai trường hợp.

Trường hợp 1: khoảng cách của p giảm. Khi đó khoảng cách của q chỉ có thể giảm. Khi vòng lặp kết thúc, hoặc gốc đã được xử lý xong và q rỗng, hoặc p là cây con phải của q và $\text{dist}(p) + 1 = \text{dist}(q)$. Nếu $\text{dist}(p) + 1 > \text{dist}(q)$, thì p nhất định là cây con trái của q ; nếu không sẽ xảy ra việc khoảng cách cây con phải của q đã giảm mà sau khi cộng 1 vẫn lớn hơn khoảng cách của q , mâu thuẫn với tính chất 3. Trong mọi trường hợp, thao tác xóa có thể dừng. Chú ý rằng mỗi lần lặp, khoảng cách của p tăng thêm 1, và trong thân vòng lặp $\text{dist}(p) + 1$ cuối cùng trở thành khoảng cách của một nút nào đó. Theo tính chất 4, mọi khoảng cách đều không vượt quá $\log n$, nên số lần lặp không vượt quá $\log n$.

Trường hợp 2: khoảng cách của p tăng. Trong trường hợp này ta chắc chắn điều chỉnh liên tục từ cây con phải đi lên, cho tới khi q rỗng hoặc p là cây con trái của q . Việc luôn đi lên từ cây con phải cho thấy số vòng lặp không vượt quá $\log n$, theo tính chất 4.

Cuối cùng, hai trường hợp trên không thể luân phiên. Sau khi việc điều chỉnh của một trường hợp kết thúc, ta đã biết một trong ba điều: q rỗng, $\text{dist}(p) + 1 = \text{dist}(q)$, hoặc p là cây con trái của q . Không tình huống nào dẫn tới trường hợp còn lại. Trực giác là: nếu cây con mới sau khi gộp làm phát sinh một chuỗi điều chỉnh khoảng cách ở các nút cha, thì chuỗi đó hoặc luôn giảm, hoặc luôn tăng, không xen kẽ.

Ta đã biết việc tạo cây con mới p tốn $O(\log n)$, và việc điều chỉnh khoảng cách đi lên cũng tốn $O(\log n)$. Vì vậy độ phức tạp xấu nhất của thao tác xóa là $O(\log n)$. Nếu cây lệch trái rất nghiêng, trong thực tế thao tác có thể nhanh hơn đáng kể.

3.6 Nhận xét

Phần này đã giới thiệu các thao tác trên cây lệch trái. Có thể thấy, với vai trò hiện thực đồng có thể gộp, các thao tác của cây lệch trái đều có hiệu năng rất tốt, trong khi độ phức tạp lập trình khá thấp. Nói cách khác, đây là một cấu trúc dữ liệu có tỷ lệ lợi ích trên chi phí cao.

Cây lệch trái là một hiện thực tốt của đồng có thể gộp vì nó nắm bắt được một số thông tin hữu ích khác trong cây nhị phân có tính chất đồng, thay vì bỏ phí thông tin đó. Theo tính chất đồng, đường đi từ gốc xuống bất kỳ nút ngoài nào đều có thứ tự. Đường đi càng dài cho thấy tính có thứ tự toàn cục của cây càng mạnh. Khác với cây cân bằng, vốn không cho phép đường đi quá dài, cây lệch trái giữ lại phần đáng kể của độ dài tự nhiên đó và đẩy nó sang trái, dùng nó để làm ngắn khoảng cách của nút và cải thiện hiệu năng. Ở đây ta không bàn một cách nghiêm ngặt; cây lệch trái cho thấy đại ý rằng từ bỏ thông tin sẵn có đồng nghĩa với hy sinh hiệu năng thuật toán.

Tài liệu gốc minh họa hai cực trị: cây lệch trái tốt nhất là một danh sách có thứ tự, xảy ra khi thao tác chèn theo thứ tự ngược và tính có thứ tự tự nhiên được giữ lại; cây lệch trái xấu nhất có dạng cân bằng, xảy ra khi thao tác chèn theo thứ tự thuận và tính có thứ tự tự nhiên bị mất.

4 Ứng dụng của cây lệch trái

4.1 Ví dụ: Dãy số, Baltic 2004

Mô tả bài toán. Cho một dãy số nguyên $a_1, a_2, \dots, a_n$. Hãy tìm một dãy không giảm

$$b_1 \leq b_2 \leq \dots \leq b_n$$

sao cho tổng độ lệch tuyệt đối

$$|a_1 - b_1| + |a_2 - b_2| + \dots + |a_n - b_n|$$

nhỏ nhất.

Quy mô dữ liệu.

$$1 \leq n \leq 10^6, \quad 0 \leq a_i \leq 2 \cdot 10^9.$$

Phân tích ban đầu. Trước hết xét hai trường hợp đặc biệt:

1. Nếu $a[1] \leq a[2] \leq \dots \leq a[n]$, nghiệm tối ưu hiển nhiên là $b[i] = a[i]$.

2. Nếu $a[1] \geq a[2] \geq \dots \geq a[n]$, nghiệm tối ưu là $b[i] = x$, với x là trung vị của dãy a .¹

Từ đó ta có ý tưởng ban đầu: chia đoạn $1 \dots n$ thành m đoạn

$$[q[1], q[2] - 1], [q[2], q[3] - 1], \dots, [q[m], q[m + 1] - 1],$$

trong đó quy ước $q[m + 1] = n + 1$. Mỗi đoạn tương ứng với một giá trị nghiệm:

$$b[q[i]] = b[q[i] + 1] = \dots = b[q[i + 1] - 1] = w[i],$$

với $w[i]$ là trung vị của

$$a[q[i]], a[q[i] + 1], \dots, a[q[i + 1] - 1].$$

Trong trường hợp đặc biệt thứ nhất, $m = n$ và $q[i] = i$. Trong trường hợp thứ hai, $m = 1$ và $q[1] = 1$. Câu hỏi là ý tưởng này có đúng không, và hiện thực ra sao.

Giả sử nghiệm tối ưu của nửa đầu $a[1], a[2], \dots, a[n]$ là $(u, u, \dots, u)$, còn nghiệm tối ưu của nửa sau $a[n + 1], a[n + 2], \dots, a[m]$ là $(v, v, \dots, v)$. Nghiệm tối ưu của toàn dãy là gì? Nếu $u \leq v$, rõ ràng nghiệm tối ưu là

$$(u, u, \dots, u, v, v, \dots, v).$$

Nếu $u > v$, giả sử nghiệm tối ưu toàn dãy là

$$(b[1], b[2], \dots, b[m]).$$

Khi đó hiển nhiên $b[n] \leq u$; nếu không, ta thay nghiệm của nửa đầu bằng $(u, u, \dots, u)$ và nghiệm toàn dãy không xấu đi. Tương tự, $b[n + 1] \geq v$. Ta sẽ dùng sự kiện sau.

Sự kiện. Với một dãy bất kỳ $a[1], a[2], \dots, a[n]$, nếu nghiệm tối ưu là $(u, u, \dots, u)$, thì trong trường hợp $u \leq u' \leq b[1]$ hoặc $b[n] \leq u' \leq u$, nghiệm

$$(b[1], b[2], \dots, b[n])$$

không tốt hơn

$$(u', u', \dots, u').$$

Chứng minh. Ta chứng minh trường hợp $u \leq u' \leq b[1]$ bằng quy nạp; trường hợp $b[n] \leq u' \leq u$ tương tự. Với $n = 1$, $u = a[1]$, mệnh đề hiển nhiên đúng.

Với $n > 1$, giả sử mệnh đề đúng cho mọi dãy có độ dài nhỏ hơn n . Xét dãy có độ dài n . Trước hết, thay $(b[1], b[2], \dots, b[n])$ bằng $(b[1], b[1], \dots, b[1])$. Thay đổi này không làm nghiệm xấu đi; nếu làm xấu đi, theo giả thiết quy nạp, trung vị w của $a[2], a[3], \dots, a[n]$ phải lớn hơn u . Khi đó nghiệm tối ưu phải có dạng

$$(u, u, \dots, u, w, w, \dots, w),$$

mâu thuẫn với giả thiết nghiệm tối ưu là hằng u . Tiếp theo thay $(b[1], b[1], \dots, b[1])$ bằng $(u', u', \dots, u')$. Vì

$$|a[1] - x| + |a[2] - x| + \dots + |a[n] - x|$$

có ý nghĩa hình học là tổng khoảng cách từ điểm x trên trục số tới các điểm $a[1], a[2], \dots, a[n]$, và $u \leq u' \leq b[1]$, tổng khoảng cách tại u' không lớn hơn tại $b[1]$. Do đó $(b[1], b[1], \dots, b[n])$ không tốt hơn $(u', u', \dots, u')$. Sự kiện được chứng minh.

¹Để thuận tiện cho thảo luận và hiện thực, trung vị trong bài này là phần tử lớn thứ $\lfloor n/2 \rfloor$ của dãy.

Trở lại lập luận trước đó. Do $b[n] \leq u$, từ sự kiện trên ta biết rằng thay

$$(b[1], b[2], \dots, b[n]) \text{ bằng } (b[n], b[n], \dots, b[n])$$

rồi thay

$$(b[n+1], b[n+2], \dots, b[m]) \text{ bằng } (b[n+1], b[n+1], \dots, b[n+1])$$

không làm nghiệm xấu đi. Nói cách khác, nghiệm tối ưu của toàn dãy là

$$(b[n], b[n], \dots, b[n], b[n+1], b[n+1], \dots, b[n+1]).$$

Xét ý nghĩa hình học của nghiệm này, nếu trung vị của toàn dãy là w , thì chọn $b[n] = b[n+1] = w$ rõ ràng cho nghiệm tối ưu toàn dãy, tức là

$$(w, w, \dots, w).$$

Mô tả thuật toán. Tiếp tục theo ý tưởng ban đầu, giả sử ta đã tìm được nghiệm tối ưu của k số đầu

$$a[1], a[2], \dots, a[k], \quad k < n,$$

dưới dạng một hàng đợi gồm m đoạn với nghiệm tương ứng

$$(w[1], w[2], \dots, w[m]).$$

Khi thêm $a[k+1]$, trước hết đưa nó vào cuối như một đoạn mới và đặt $w[m+1] = a[k+1]$. Sau đó liên tục kiểm tra hai đoạn cuối. Nếu $w[m] > w[m+1]$, ta gộp hai đoạn cuối và tìm nghiệm tối ưu của đoạn mới, tức là trung vị của các phần tử a có chỉ số nằm trong đoạn đó. Lặp quá trình gộp cho tới khi

$$w[1] \leq w[2] \leq \dots \leq w[m],$$

rồi xử lý số tiếp theo.

Tính đúng đắn của thuật toán đã được lập luận ở trên. Bây giờ ta xét lựa chọn cấu trúc dữ liệu. Thuật toán cần hai thao tác: gộp hai tập có thứ tự và truy vấn trung vị trong một tập có thứ tự. Có nhiều cấu trúc dữ liệu hỗ trợ tương đối tốt hai thao tác này. Ví dụ rõ ràng là cây tìm kiếm nhị phân (BST), với truy vấn $O(\log n)$, nhưng thao tác gộp không lý tưởng; nếu dùng gộp theo kích thước, tổng thời gian là $O(n \log^2 n)$.

Có lựa chọn tốt hơn không? Phân tích sâu hơn cho thấy thao tác gộp chỉ xảy ra khi trung vị trong một đoạn lớn hơn trung vị trong đoạn sau. Nói cách khác, sau khi một đoạn gộp với đoạn sau, trung vị của đoạn đó không tăng. Vì vậy ta có thể dùng đồng lớn để duy trì trung vị trong mỗi đoạn. Khi số phần tử trong đồng lớn hơn một nửa số phần tử của đoạn, xóa đỉnh đồng. Như vậy đồng luôn chứa nửa nhỏ hơn của các phần tử trong đoạn, và đỉnh đồng chính là trung vị của đoạn. Do cần hoàn thành thao tác gộp một cách hiệu quả, cây lệch trái là lựa chọn lý tưởng.² Truy vấn trên cây lệch trái tốn $O(1)$; xóa và gộp đều tốn $O(\log n)$. Số thao tác truy vấn và gộp nhỏ hơn n , số thao tác xóa không vượt quá $n/2$ vì xóa chỉ xảy ra khi gộp hai đồng có số phần tử lẻ. Do đó hiện thực bằng cây lệch trái đưa độ phức tạp về $O(n \log n)$.

Nhận xét. Quá trình giải bài này rất gợi mở. Khi áp dụng cây lệch trái, ta thường có cảm giác không biết bắt đầu từ đâu, thậm chí bài toán tưởng như không liên quan tới cây lệch trái. Nhưng nếu phân tích sâu và chuyển đổi thích hợp, vấn đề có thể được giải quyết. Điều đó đòi hỏi tư duy linh hoạt và cảm giác bài toán tốt.

So với cách dùng BST ở trên, lời giải bằng cây lệch trái có độ phức tạp thời gian và độ phức tạp lập trình thấp hơn. Điều này không có nghĩa cây lệch trái luôn là lời giải tốt nhất. Riêng bài này có nhiều cách giải, và ngay trong nhóm đồng có thể gộp cũng có thể dùng nhiều cấu trúc dữ liệu khác nhau. Tuy vậy, cây lệch trái vẫn có ưu thế nhất định so với chúng; phần sau thảo luận chi tiết hơn.

²Phần trước giới thiệu cây lệch trái dưới dạng đồng nhỏ; trong bài này chỉ cần sửa tính chất so sánh một chút là có thể hiện thực đồng lớn.

5 So sánh cây lệch trái với các đồng có thể gộp khác

Ta biết cây lệch trái là một cách hiện thực đồng có thể gộp. Nhưng vì sao nên dùng cây lệch trái? Nó có ưu điểm gì so với các đồng có thể gộp khác? Phần này giới thiệu đặc điểm của nhiều loại đồng có thể gộp và so sánh chúng.

5.1 Biến thể của cây lệch trái: đồng nghiêng

Đồng nghiêng (*skew heap*) là một biến thể của cây lệch trái. Nó là một cây nhị phân có thứ tự đồng, nhưng không thỏa tính chất lệch trái; nói cách khác, đồng nghiêng không có khái niệm “khoảng cách” và không cần ghi khoảng cách ở bất kỳ nút nào. Về mặt cấu trúc, mọi cây lệch trái đều là đồng nghiêng, nhưng chiều ngược lại không đúng.

Tương tự cây lệch trái, các thao tác trên đồng nghiêng cũng được xây dựng dựa trên thao tác gộp. Vì vậy ở đây chỉ giới thiệu thao tác gộp; các thao tác khác có thể làm tương tự cây lệch trái.

Quá trình đệ quy khi gộp đồng nghiêng giống hệt cây lệch trái. Giả sử cần gộp hai đồng nghiêng A và B , và gốc của A nhỏ hơn gốc của B . Ta lấy gốc của A làm gốc của đồng mới, rồi gộp cây con phải của A với B . Vì việc gộp luôn đi theo đường phải ngoài cùng, sau khi gộp, độ dài đường phải ngoài cùng của đồng mới chắc chắn tăng, ảnh hưởng tới hiệu quả lần gộp sau. Cây lệch trái giải quyết bằng cách kiểm tra khoảng cách của các nút trên đường phải ngoài cùng và hoán đổi hai cây con để đường đó rất ngắn. Đồng nghiêng không ghi khoảng cách, vậy phải duy trì đường phải ngoài cùng như thế nào? Cách làm là đi từ dưới lên trên dọc theo đường gộp và tại mỗi nút đều hoán đổi hai cây con.

Mã giả gộp đồng nghiêng:

```
Function Merge(A, B)
  If A = NULL Then return B
  If B = NULL Then return A
  If key(B) < key(A) Then swap(A, B)
  right(A) <- Merge(right(A), B)
  swap(left(A), right(A))
  return A
End Function
```

Cách duy trì này của đồng nghiêng cũng hiệu quả. Bằng việc liên tục hoán đổi hai cây con, đồng nghiêng đẩy đường phải ngoài cùng sang trái. Có thể chứng minh thao tác gộp của đồng nghiêng có độ phức tạp khấu hao $O(\log n)$. Phần phân tích chi tiết không được trình bày ở đây; độc giả quan tâm có thể tham khảo tài liệu liên quan.

Các thao tác khác của đồng nghiêng tương tự cây lệch trái, nên độ phức tạp khấu hao của chúng cũng giống cây lệch trái. Về không gian, do đồng nghiêng không ghi khoảng cách của nút, nó cần ít không gian hơn một chút. Về độ phức tạp lập trình, cả hai đều rất thấp, nhưng mã của đồng nghiêng ngắn hơn một chút. Nhược điểm của đồng nghiêng là độ phức tạp của một lần gộp có thể suy thoái tới $O(n)$, tuy điều này thường không ảnh hưởng tới tổng độ phức tạp của thuật toán.

Nhìn chung, đồng nghiêng là biến thể của cây lệch trái và không có bên nào vượt trội tuyệt đối. Nơi nào đồng nghiêng dùng được, cây lệch trái cũng có thể hiện thực tốt thuật toán. Quan hệ giữa đồng nghiêng và cây lệch trái có thể ví như quan hệ giữa cây splay và cây AVL, chỉ khác là độ phức tạp lập trình của hai đồng trước không chênh lệch nhiều.

5.2 So sánh với đồng nhị phân

Đồng nhị phân có lẽ là hàng đợi ưu tiên được dùng nhiều nhất. Nó đơn giản, hiệu quả và có phạm vi ứng dụng rộng. So với cây lệch trái, mọi thao tác của đồng nhị phân, trừ thao tác gộp, đều có cùng độ phức tạp thời gian; trong thử nghiệm thực tế, đồng nhị phân thường nhanh hơn cây lệch trái. Về không gian, vì đồng nhị phân là cây nhị phân đầy đủ và có thể biểu diễn bằng mảng, nó tiết kiệm được không gian con trỏ trái phải; ở điểm này cây lệch trái chịu thiệt.

Sau khi nói về ưu điểm của đồng nhị phân, hai nhược điểm sau cũng không thể bỏ qua:

1. Sau các thao tác chèn, xóa, v.v., vị trí vật lý của phần tử trong đồng nhị phân thay đổi.
2. Thao tác gộp của đồng nhị phân quá chậm, với độ phức tạp $O(n)$.

Khi bản thân phần tử chiếm nhiều không gian, việc thường xuyên di chuyển vị trí vật lý của phần tử ảnh hưởng tới hiệu suất thời gian. Ngoài ra, đôi khi ta cần nắm vị trí vật lý của một phần tử để truy cập hoặc sửa nó, chẳng hạn khi hiện thực thao tác xóa nút đã biết. Khi đó nhược điểm thứ nhất của đồng nhị phân gây khó khăn, buộc ta phải bổ sung con trỏ hoặc cơ chế tương tự. Còn hiệu quả gộp thấp của đồng nhị phân bắt nguồn từ chính tính chất của nó; gộp theo kích thước có thể là một tối ưu trong nhiều tình huống, nhưng kết quả vẫn chưa thỏa đáng.

Ưu thế không gian của đồng nhị phân cũng không tuyệt đối. Biểu diễn bằng mảng không phù hợp với mọi trường hợp. Chẳng hạn khi cần duy trì động nhiều hàng đợi ưu tiên, đồng nhị phân buộc phải dùng cách biểu diễn con trỏ truyền thống; khi đó không gian của đồng nhị phân và cây lệch trái gần như tương đương.

Các đặc điểm trên quyết định rằng đồng nhị phân không phù hợp để hiện thực đồng có thể gộp. Khách quan mà nói, bài thi cần hiện thực đồng có thể gộp không quá nhiều. Nhưng khi gặp loại bài đó hoặc một số tình huống đặc biệt, cây lệch trái sẽ là lựa chọn tốt hơn đồng nhị phân.

5.3 So sánh với các đồng có thể gộp khác

Như đã nhắc tới, đồng có thể gộp có thể được hiện thực bằng nhiều cấu trúc dữ liệu; cây lệch trái không phải lựa chọn duy nhất. Đồng nhị thức và đồng Fibonacci đều là các đồng có thể gộp có hiệu quả thời gian rất tốt.

Đồng nhị thức là một rừng gồm nhiều cây nhị thức có độ sâu khác nhau. Cây nhị thức B_k có độ sâu k được tạo bằng cách nối hai cây nhị thức B_{k-1} tại gốc, có 2^k nút và thỏa tính chất đồng. Hình thức rừng các cây nhị thức có thể xem như biểu diễn nhị phân của tổng số nút n ; việc gộp hai đồng nhị thức có thể xem như phép cộng nhị phân, một góc nhìn rất gợi mở. Tương tự cây lệch trái, các thao tác trên đồng nhị thức cũng dựa vào thao tác gộp. Cấu trúc và tính chất của đồng nhị thức giúp nó gộp rất hiệu quả. So với cây lệch trái, tuy độ phức tạp giống nhau, trong thực tế đồng nhị thức thường nhanh hơn. Nhưng thao tác lấy nút nhỏ nhất của đồng nhị thức phải kiểm tra gốc của mọi cây nhị thức, nên độ phức tạp cao hơn cây lệch trái.³

Đồng Fibonacci là một cấu trúc dữ liệu rất phức tạp. Giống đồng nhị thức, nó cũng gồm một tập cây có thứ tự đồng. Điểm khác là các cây của đồng Fibonacci không nhất thiết là cây nhị thức, các cây không có thứ tự, và các nút anh em trong cây được liên kết bằng danh sách vòng hai chiều. Thao tác chèn và gộp của đồng Fibonacci chỉ đơn giản nối hai danh sách gốc lại với nhau, nên hai thao tác này nhanh hơn mọi đồng có thể gộp khác. Nhưng khi xóa, ta phải bảo trì đồng Fibonacci bằng cách gộp mọi cây có cùng bậc; bước này đặc biệt phức tạp và là bước chậm nhất.

³Có một cách giải quyết: luôn ghi lại nút nhỏ nhất và chỉ cập nhật bản ghi đó khi chèn, xóa hoặc gộp. Nhờ vậy thao tác lấy nút nhỏ nhất của đồng nhị thức có thể giảm về $O(1)$.

Độc giả quan tâm tới đồng nhị thức và đồng Fibonacci có thể tham khảo tài liệu liên quan. Bảng sau liệt kê độ phức tạp thời gian của các thao tác, cùng nhu cầu không gian và độ phức tạp lập trình.⁴

Mục	Đồng nhị phân	Cây lệch trái	Đồng nhị thức	Đồng Fibonacci
Xây dựng	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Chèn	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(1)$
Lấy nhỏ nhất	$O(1)$	$O(1)$	$O(\log n)$	$O(1)$
Xóa nhỏ nhất	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Xóa nút bất kỳ	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Gộp	$O(n)$	$O(\log n)$	$O(\log n)$	$O(1)$
Không gian	Ít nhất	Khá ít	Trung bình	Nhiều
Lập trình	Thấp nhất	Thấp	Cao	Rất cao

Từ bảng có thể thấy đồng Fibonacci có độ phức tạp thời gian rất thấp. Nếu không cần xóa thường xuyên, dùng đồng Fibonacci để hiện thực đồng có thể gộp có thể giảm độ phức tạp thuật toán. Nhưng trong thực tế, xóa thường là thao tác quan trọng, và thao tác xóa của đồng Fibonacci chậm hơn các đồng khác trong bảng. Nhu cầu không gian của nó cũng lớn đến mức khiến người ta e ngại. Tệ hơn nữa, độ phức tạp lập trình của đồng Fibonacci quá cao; dùng nó trong thi lập trình thường là không sáng suốt.

Đối với đồng nhị thức, tuy hiệu quả thời gian của các thao tác đều rất tốt, nhu cầu không gian và độ phức tạp lập trình vẫn không bằng cây lệch trái. So với cây lệch trái, ưu thế thời gian của đồng nhị thức rất nhỏ. Thay đồng nhị thức bằng cây lệch trái đúng là hy sinh một ít hiệu năng, nhưng đổi lại là mã ngắn gọn, dễ hiện thực và dễ gỡ lỗi. Trong kì thi có thời gian hạn chế, cây lệch trái rõ ràng là lựa chọn tốt hơn.

Cuối cùng, cần nhận ra rằng dù một số thao tác của đồng nhị thức và đồng Fibonacci có độ phức tạp thấp hơn cây lệch trái, trong ứng dụng thực tế, các thao tác có độ phức tạp cao thường trở thành nút thắt thuật toán. Ví dụ trong bài *Dãy số* ở trên, nếu đổi sang đồng nhị thức hoặc đồng Fibonacci, tổng độ phức tạp vẫn là $O(n \log n)$, không làm giảm bậc độ phức tạp nhưng lại làm tăng độ khó hiện thực.

6 Kết luận

Đến đây ta đã có cái nhìn khá sâu về cây lệch trái. Về hiệu quả thời gian, nó không bằng đồng nhị thức và đồng Fibonacci; về hiệu quả không gian, nó không bằng đồng nhị phân. Nhìn qua, cây lệch trái dường như không có điểm độc đáo và có vẻ là một cấu trúc dữ liệu khá bình thường. Nhưng đó chính là ưu thế của nó. Thời gian, không gian và độ phức tạp lập trình thường mâu thuẫn với nhau. Đồng Fibonacci có độ phức tạp thời gian thấp nhất nhưng độ phức tạp lập trình khó chấp nhận; đồng nhị phân có không gian và lập trình rất thấp nhưng độ phức tạp gộp lại là điểm yếu chí mạng. Cây lệch trái điều hòa tốt ba yếu tố này và không có khiếm khuyết về vị trí lưu trữ như đồng nhị phân. Vì vậy phạm vi áp dụng của cây lệch trái rất rộng. Nó không chỉ hiện thực đồng có thể gộp một cách hiệu quả và thuận tiện, mà còn có thể thay thế đồng nhị phân trong nhiều hàng đợi ưu tiên; nhiều lúc còn tiện hơn đồng nhị phân.

Lời cảm ơn

Tác giả cảm ơn bạn Yu Jiangwei và thầy Ruan Zhiyuan vì sự giúp đỡ nhiệt tình và những góp ý chỉnh sửa.

⁴Trong bảng, độ phức tạp của hai thao tác xóa trên đồng Fibonacci là độ phức tạp khá hao. Thao tác chèn của đồng nhị thức có độ phức tạp trung bình $O(1)$; bảng ghi độ phức tạp xấu nhất.

Tài liệu tham khảo

1. Fu Qingxiang, Wang Xiaodong. *Thuật toán và cấu trúc dữ liệu*, bản thứ hai. Nhà xuất bản Công nghiệp Điện tử.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, second edition. The MIT Press.
3. Mark Allen Weiss. *Data Structures and Algorithm Analysis in C*, second edition. Pearson Education.

A Nguyên đề bài ví dụ Dãy số

SEQUENCE

Phát biểu ngắn. Cho một dãy số. Nhiệm vụ của bạn là xây dựng một dãy tăng xấp xỉ dãy đã cho tốt nhất. Dãy xấp xỉ tốt nhất là dãy có tổng độ lệch so với dãy đã cho nhỏ nhất.

Phát biểu chính xác hơn. Cho dãy số $t_1, t_2, \dots, t_N$. Hãy xây dựng dãy số tăng

$$z_1 < z_2 < \dots < z_N.$$

Tổng

$$|t_1 - z_1| + |t_2 - z_2| + \dots + |t_N - z_N|$$

phải nhỏ nhất có thể.

Dữ liệu vào. Dòng đầu của tệp `seq.in` chứa số nguyên N ($1 \leq N \leq 1000000$). Mỗi dòng trong N dòng tiếp theo chứa một số nguyên: phần tử của dãy đã cho. Dòng thứ $K + 1$ chứa t_K . Mọi phần tử thỏa

$$0 \leq t_K \leq 2000000000.$$

Dữ liệu ra. Dòng đầu của tệp `seq.out` phải chứa một số nguyên duy nhất: tổng độ lệch nhỏ nhất có thể. Mỗi dòng trong N dòng tiếp theo phải chứa một số nguyên: phần tử tương ứng của dãy xấp xỉ tốt nhất. Nếu có nhiều nghiệm, chương trình có thể in bất kỳ dãy nào có tổng độ lệch nhỏ nhất.

Ví dụ.	seq.in	seq.out
	7	13
	9	6
	4	7
	8	8
	20	13
	14	14
	15	15
	18	18

B Chương trình lời giải bằng cây lệch trái

```
PROGRAM Seq_LeftistTree;
```

```

Type
node=
record
    dist:byte;
    key,left,right:longint;
end;
VaR
nd:array[0..1000000]of node;
stk,q:array[0..1000000]of longint;
n,cl,i:longint;
Function merge(a,b:longint):longint;
var
    c:longint;
begin
    if (a=0)or(b<>0)and(nd[a].key<nd[b].key) then
    begin
        c:=a;
        a:=b;
        b:=c;
    end;
    merge:=a;
    if b=0 then exit;
    nd[a].right:=merge(nd[a].right,b);
    if nd[nd[a].left].dist<nd[nd[a].right].dist then
    begin
        c:=nd[a].left;
        nd[a].left:=nd[a].right;
        nd[a].right:=c;
    end;
    nd[a].dist:=nd[nd[a].right].dist+1;
end;
Procedure print;
var
    tot,i,j:longint;
begin
    tot:=0;
    for i:=1 to cl do
        for j:=q[i-1]+1 to q[i] do
            inc(tot,abs(nd[j].key-nd[stk[i]].key));
        writeln(tot);
    end;
BeGiN
assign(input,'seq.in');
reset(input);
assign(output,'seq.out');
rewrite(output);
readln(n);

```

```

fillchar(nd,sizeof(nd),0);
cl:=0;
q[0]:=0;
for i:=1 to n do
begin
  readln(nd[i].key);
  dec(nd[i].key,i);
  inc(cl);
  stk[cl]:=i;
  q[cl]:=i;
  while (cl>1)and(nd[stk[cl]].key<nd[stk[cl-1]].key) do
  begin
    dec(cl);
    stk[cl]:=merge(stk[cl],stk[cl+1]);
    if odd(q[cl+1]-q[cl]) and odd(q[cl]-q[cl-1]) then
      stk[cl]:=merge(nd[stk[cl]].left,nd[stk[cl]].right);
    q[cl]:=q[cl+1];
  end;
end;
print;
close(output);
EnD.

```